

```
(%i2) {dog, cat, donkey, cow, buffalo};
(%o2)          {buffalo, cat, cow, dog, donkey}
(%i3) {Kiran, Kasturi, Karan};
(%o3)          {Karan, Kasturi, Kiran}
(%i4) {6, horse, sapphire};
(%o4)          {6, horse, sapphire}
(%i5) {axe, knife, spear, axe, scissor};
(%o5)          {axe, knife, scissor, spear}
(%i6) quit();
```

Note that as the order of items in the set doesn't matter, Maxima internally keeps them sorted, and hence displayed accordingly, as in the above examples. Also, note the last example—the duplicates are treated as a single item.

Sets can also be created from ordered lists using *setify*. Members of sets could be expressions, but may not be automatically simplified. Check out the following:

```
$ maxima -q
(%i1) setify([x, y, z]);
(%o1)          {x, y, z}
(%i2) [x, y, z, x]; /* Ordered list */
(%o2)          [x, y, z, x]
(%i3) setify([x, y, z, x]);
(%o3)          {x, y, z}
(%i4) string((x^2 - 1, (x + 1) * (x - 1)));
(%o4)          {(x-1)*(x+1),x^2-1}
```

string() has been used in %i4, to just have the output on a single line. But the important thing to note is that though the two items of the list are mathematically identical, they have been preserved as two distinct items and thus do not form a set in the real sense. Such cases can be actually formed into a set by simplifying the individual items of the set using a corresponding simplification function, e.g., *rat()* for rational expressions. And operating any function on every item of a set can be achieved using *map()*. Here's an example to *get all those straight*, continuing from the above:

```
(%i5) string(map(rat, {x^2 - 1, (x + 1) * (x - 1)}));
(%o5)          {x^2-1}
(%i6) string(rat((x + 1) * (x - 1)));
(%o6)          x^2-1
(%i7) quit();
```

%i6 and %o6 shown above are just to demonstrate how *rat()* works. I know you are still wondering what this weird *map()* is and how it works. So, here are a few more examples:

```
$ maxima -q
(%i1) trigreduce(2 * sin(x) * cos(x));
(%o1)          sin(2 x)
(%i2) sin(2 * x), 2 * sin(x) * cos(x); /* Identical items */
(%o2)          {2 cos(x) sin(x), sin(2 x)}
```

```
(%i3) map(trigreduce, {sin(2 * x), 2 * sin(x) * cos(x)});
(%o3)          {sin(2 x)}
(%i4) string({apple / fruit + mango / fruit, (apple + mango) / fruit});
(%o4)          {(mango+apple)/fruit, mango/fruit+apple/fruit}
(%i5) string(map(rat, {apple / fruit + mango / fruit, (apple + mango) / fruit}));
(%o5)          {(mango+apple)/fruit}
(%i6) quit();
```

In fact, the power of *map()* lies in its ability to take a function created on-the-fly, using the *lambda* notation. Here are a few examples to demonstrate *lamda()* first, and then *map()* using *lambda()*:

```
$ maxima -q
(%i1) f: lambda([x], x^3)$
(%i2) f(5);
(%o2)          125
(%i3) lambda([x], x^3)(5);
(%o3)          125
(%i4) lambda([x, y], x+y)(4, 6);
(%o4)          10
(%i5) map(f, {0, 1, 2, 3});
(%o5)          {0, 1, 8, 27}
(%i6) map(lambda([x], x^3), {0, 1, 2, 3});
(%o6)          {0, 1, 8, 27}
(%i7) map(lambda([x, y], x+y), {a}, {3});
(%o7)          {a + 3}
(%i8) map(lambda([x], x^4), {-2, -1, 0, 1, 2});
(%o8)          {0, 1, 16}
(%i9) map(g, {-2, -1, 0, 1, 2});
(%o9)          {g(- 2), g(- 1), g(0), g(1), g(2)}
(%i10) quit();
```

lambda() takes two arguments. First, a list of arguments of the function being defined, and second, the expression for the return value of the function using those arguments. %i1 defines a function 'f' with one argument, returning its cube. %i2 calls *f()*. However, the whole point of using *lambda* is to use it without defining an explicit function like *f()*. So, %i3 and %i4 demonstrate exactly that. %i6, %i7 and %i8 show how to use *lambda()* with *map()*. Note the elimination of duplicates in %o8. %i9 is another example of *map()*.

Basic set operations

Now, that's enough about the creation of different varieties of sets. Let's do some set operations. For starters, the union of sets is defined as a set with the items of all the sets, the intersection of sets is defined as a set with items common to all the sets, and the difference of two sets is defined as a set with items from the first set, but not in the